

Arab Open University - Computer Science



Faculty of Computer Studies

Information Technology and Computing Department

AI Automated Trading System

Ahmed Sherif Mohamed Sabry Hassanein - 21512145

TM471: Final Year Project, May 2025

Supervisor: *Prof. Mahmoud Atallah*

Supervisor: *Dr. Sana*

Table of Contents	Page
Abstract	4
Acknowledgements	5
Overview	6
Chapter 1: Introduction.	7
1.1 Background and Motivation	7
1.2 Objectives	7
1.3 Problem Statement	8
1.4 Scope of the Project	8
1.5 Deliverables	9
1.6 Target Customer and Audience	9
1.7 Project Plan	10
1.7.1 Objectives and Milestones	10
1.7.2 Project Timeline	11
1.7.3 Resource Allocation	12
1.7.4 Risk Management	12
Chapter 2: Literature Review	13
2.1 Related Works	13
2.2 Advantages of the Proposed System	14
2.3 Disadvantages of Existing Solutions	15
Chapter 3: Requirements and Analysis	16
3.1 Functional Requirements	16
3.2 Non-Functional Requirements	16
3.3 Software and Hardware Requirements	17
3.4 Initial System Design	18
3.5 Feature insights and future enhancements	24
Chapter 4: Design, implementation, and testing	25
4.1 Overview	25
4.2 Software Development Approach	25
4.2.1 Used Approach	25
4.2.2 Justification of the used approach	25
4.2.3 Phases of the chose model approach	26
4.2.4 Why it is better than other approaches	26
4.2.5 Alternative approach	26

4.3 Changes in design	27
4.3.1 Changes in the functional requirements	27
4.3.2 Changes in the non-functional requirements	27
4.3.3 Changes in diagrams	27
4.3.4 Changes in software requirements	27
4.3.5 Software modeling tools used	28
4.3.6 Specifications that distinguish this software from the other software	28
4.4 Implementation	28
4.4.1 User interface	28
4.4.2 Code	31
4.4.3 Database	32
4.5 Issues and problems faced in the project	32
4.6 Testing	33
4.6.1 White box testing	33
4.6.2 Black box testing	33
4.6.3 Unit testing	33
4.6.4 System testing	33
4.6.5 Acceptance testing	34
4.6.6 Validation and Verification	34
4.7 Summary of the chapter	34
Chapter 5: Results and discission	35
5.1 Overview about this chapter	35
5.2 Critical discussion	35
5.3 Skills acquired from the project	36
5.4 Development goals and results	37
5.4.1 Finding and results	37
5.4.2 Project goals	38
5.5 Future work	39
5.5.1 New area of investigation	39
5.5.2 Features were not completed due to the time constrains	39
5.6 Summary of the chapter	40
Chapter 6: Conclusion	41
References	43

Abstract

This project focuses on the development of an AI-powered trading system designed to assist traders in making informed decisions by predicting stock price movements. The system leverages machine learning techniques to analyze historical market data, providing actionable insights through automated buy/sell signals. By offering traders predictions and allowing them to approve or reject trade actions, the system enhances decision-making speed and accuracy while reducing human errors and emotional bias. The system's user-friendly design ensures that both novice and experienced traders can benefit from its features.

Acknowledgements

The Accomplishment completion of this project required advice and support from many individuals, to whom I would like to express my sincere gratitude.

I would like to express our sincere thanks to *Prof. Eid Emary*, Dean of Faculty, for providing us with all the required facilities.

I would like to express our gratitude and appreciation to *Prof. Mahmoud Atallah* for his valuable advice, conscientious guidance, and encouragement throughout the whole project.

Lastly, I would like to express my heartfelt thanks to my parents and family, whose unwavering support and encouragement have been a constant source of strength throughout my academic journey. I hope to be able to give back even a small fraction of the love and support they have always shown me.

Overview

The project aims to develop an automated trading system that integrates artificial intelligence to analyze financial data and provide recommendations. The system is intended for traders seeking to improve their trading strategies through data-driven insights. It processes large amounts of historical market data and generates predictions that guide decision-making.

The final decisions on executing trades remain with the user, ensuring a balance between automation and human oversight. The project will also focus on evaluating different machine learning models to determine the most effective approach for predicting stock prices.

Chapter 1: Introduction

1.1 Background and Motivation

The financial market changes very quickly, and traders need to make decisions both fast and accurately. Traditional trading methods, which depend mostly on human judgment, often lead to mistakes and emotional decisions, leading to missed opportunities or financial losses.

Machine Learning (ML), a subset of Artificial Intelligence (AI), offers a solution to these challenges. ML models can analyze large datasets quickly, recognize patterns, and make predictions without being directly programmed. This capability has made ML increasingly popular in fields like finance, where it is used to enhance decision-making and automate complex processes.

The motivation behind this project is to use machine learning to help traders make better predictions about stock prices. By using AI for predictions, the system allows traders to receive automated recommendations while still giving them control over final decisions. This helps traders respond faster and reduces mistakes caused by emotions.

1.2 Objectives

The project aims to develop an AI-based system that uses machine learning to analyze historical market data. By identifying patterns and trends, the system can predict stock price movements and provide helpful insights.

The system will generate buy and sell signals based on predictions from machine learning algorithms. However, the final approval will be made by the trader. This approach gives traders useful recommendations while still keeping them in control. Additionally, the system will offer an interface that allows users to monitor, review, and interact with predictions. This supports more confident decisions, better risk management, and an overall improvement in trading performance.

1.3 Problem Statement

Trading in financial markets is a challenging task because prices change very quickly and a large amount of data needs to be analyzed in a short time. Human traders often struggle to keep up with this fast pace, which can lead to slow decisions, mistakes, or missed opportunities. Emotional reactions can also affect judgment, making trading less reliable.

The main problem this project addresses is the difficulty traders face in making fast and accurate decisions when dealing with large and constantly changing market data. Traditional methods that rely only on human analysis are not always effective in such a fast-moving environment.

This project proposes a solution by developing an automated trading system that uses machine learning to analyze historical market data and predict stock price movements. The system will generate buy/sell signals based on these predictions to assist the trader, but the final approval will still be made by the human user. This approach helps improve trading speed and accuracy, while reducing the risk of emotional or rushed decisions.

1.4 Scope of the Project

This project focuses on solving the challenges faced by traders in making quick and accurate trading decisions. The system will use machine learning models to analyze historical market data and identify patterns that help predict future stock prices. Based on these predictions, the system will generate buy and sell signals to assist the trader in making decisions.

To maintain trader involvement and control, the system will provide recommendations, but the final approval to execute a trade will be made by the trader. This helps reduce emotional bias while still keeping the human in charge.

The project will also include the development of a user-friendly interface. This interface will allow traders to easily access the system's predictions, review the generated signals, and approve or reject suggested trades. The goal is to improve decision-making speed, accuracy, and confidence while reducing the risk of mistakes in fast-changing market environments.

1.5 Deliverables.

This project will result in the following key deliverables:

- A web application that allows traders to view predictions and buy/sell signals.
- Trained machine learning models for stock price prediction, integrated with a recommendation system that suggests actions based on predictions, pending trader approval.
- A comprehensive final report detailing the methodology, model evaluation, training process, and system design, including UML diagrams.

1.6 Target Customer and Audience

This project is designed for people who trade in the stock market. It helps them make faster and better decisions using AI predictions and buy/sell signals.

The system is useful for:

- **New and individual traders** who want help understanding market movements and improving their decisions using AI-powered insights.
- **Experienced traders and financial analysts** who want smarter tools to improve their strategies.
- **Financial companies** that need AI-based systems to enhance their trading performance.

The system is user-friendly and does not require advanced technical knowledge, making it accessible to a wide range of users in the finance and trading sectors.

1.7 Project Plan

1.7.1 Objectives and Milestones

Objectives:

- Develop an automated AI-based stock trading system that predicts stock prices using machine learning techniques like LSTM and Random Forest.
- Design a user-friendly interface for traders to interact with the system.
- Ensure that the system generates buy/sell signals based on predictions and allows traders to approve or reject them.
- Test and validate the system using historical stock data to ensure its reliability in real market scenarios.

Milestones:

1. Milestone 1: Proposal Submission and Review

- Submit project proposal (including objectives and methodology).
- Feedback and revisions.

2. Milestone 2: System Analysis and Design

- Completion of system design documentation.
- Approval of the initial design and architecture.

3. Milestone 3: Development and Implementation

- Completion of the initial implementation of machine learning models and user interface.
- Start of integration of predictions and signal generation.

4. Milestone 4: Testing and Validation

- Testing of the system using live data or simulated environments.
- Bug fixing and improvements.

5. Milestone 5: Final Report and Presentation

- Final report submission, including all system details, evaluation, and results.
- Final presentation and demo of the working system.

1.7.2 Project Timeline

AI Automated Trading System

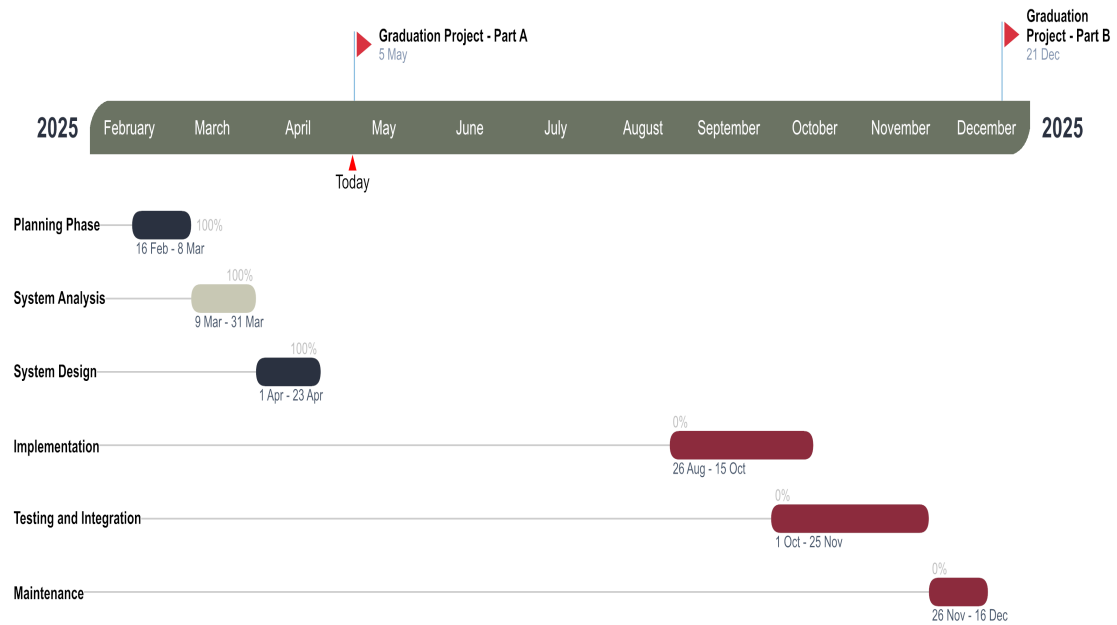


Figure 1: Project Time Plan "Gant Chart"

1.7.3 Resource Allocation

For this project, resources can be allocated as follows:

- **Team Members:**
 - Primary developer and researcher (ML models, testing, integration).
 - Project Advisor/Professor: Provides guidance and feedback on progress.
- **Tools and Software:**
 - **Programming Languages:** Python (for machine learning models), JavaScript or Python (for web app development).
 - **Frameworks:** Streamlit (for web app), Scikit-learn, TensorFlow (for machine learning models).
 - **Databases:** SQLite or MySQL (for storing generated signals, trades, and historical data).
 - **Development Environment:** Jupyter notebooks, Visual Studio Code, or Google Colab.
 - **Data:** Historical stock data from sources like Yahoo Finance, Kaggle.
- **Hardware:**
 - **Computers:** Personal Laptop.
 - **Cloud/Local Servers:** If additional computing resources are needed for model training.

1.7.4 Risk Management

Risk	Impact	Solution
Data not available	Medium	Use yfinance Library or CSV backup
Model accuracy too low	High	Try other models or tuning
App performance issues	Medium	Optimize code and model size
Time delay in development	High	Stick to timeline and plan early

By identifying these potential risks early, you can take proactive steps to mitigate them and keep the project on track.

Chapter 2: Literature Review

2.1 Related Work

In this section, we'll review existing projects or research related to AI and machine learning in stock market trading. These studies have explored ways to use technology to make trading more efficient.

2.1.1 Stock Price Prediction Using Machine Learning Models

- **Description:** This study focuses on applying machine learning models, specifically LSTM (Long Short-Term Memory) networks, to predict stock prices based on historical data. The authors show that LSTM can be more effective than traditional methods due to its ability to learn from past sequences and trends.
- **Key Contribution:** The system uses historical stock prices to predict future price movements. However, it lacks a decision-making component for traders. The focus is on prediction accuracy, not on providing an interactive interface or trader control.
- **Limitations:** The model provides no mechanism for traders to influence predictions or manage risks interactively.

2.1.2 Automated Trading System for Financial Markets

- **Description:** This research introduces a fully automated trading system that uses machine learning to analyze market data and automatically generate buy and sell signals. The system can execute trades without human intervention, reducing the time spent on decision-making.
- **Key Contribution:** By automating both the analysis and execution of trades, this system helps increase trading speed. However, it removes the human decision-making aspect entirely, which could be problematic for traders who prefer to have control over their trades.
- **Limitations:** Lack of trader involvement in the final decision may result in missed opportunities where human judgment could have been more effective. Traders may not trust the automation to make the right choices in volatile market conditions.

2.1.3 Enhancing Stock Trading Strategies with AI (Hybrid Models and AI-Driven Insights)

- **Description:** This work combines machine learning models like Random Forest and Support Vector Machines (SVM) to enhance stock trading strategies. The system focuses on optimizing trading strategies by integrating multiple models to make smarter predictions.
- **Key Contribution:** The system provides traders with more accurate predictions and better trading strategies. However, it still lacks an intuitive user interface for traders to interact with predictions, and its complexity can be a barrier for beginner users.
- **Limitations:** Although more advanced and accurate, the complexity of the system means it is less accessible for novice traders, and the system does not include an interface that would allow traders to make decisions based on the predictions.

These related works showcase different approaches to using AI in stock market trading but differ in terms of user control, ease of use, and the range of trading functions. Our project aims to fill the gap by offering a system that not only predicts stock prices but also allows traders to approve trades based on those predictions.

2.2 Advantages of the Proposed System

The solution we propose offers several advantages over existing systems:

- **User Control:** Our system allows traders to approve trades, which helps avoid emotional decisions and gives the trader more control over their trades.
- **Faster Decision-Making:** With machine learning models that analyze large datasets quickly, the system can help traders make faster decisions, reducing the risk of missing opportunities.
- **User-Friendly Interface:** Our system is designed with simplicity in mind, making it easy for both beginners and experienced traders to use without needing advanced technical knowledge.
- **Enhanced Accuracy:** By using machine learning models like LSTM and Random Forest, our system provides more accurate predictions, helping traders make better-informed decisions.

2.3 Disadvantages of the Existing Solutions

While the existing systems offer valuable tools for trading, they have some limitations:

- **Limited Trader Control:** Many automated systems make trading decisions without involving the trader, which may not be ideal for those who prefer to remain involved in the decision-making process.
- **Complexity:** Some existing systems are difficult to use, requiring advanced technical knowledge or training, which limits their accessibility for beginner traders.
- **Limited Customization:** Some systems do not allow traders to adjust predictions or signals based on their preferences, reducing the flexibility of the tool.

Limitations of the Proposed System

While our system offers several advantages, there are potential drawbacks as well:

- **Dependence on Historical Data:** Machine learning models are only as good as the data they are trained on. The system's predictions are based on past market trends, which might not always predict future market conditions accurately.
- **Emotional Bias in Decision-Making:** While the system reduces emotional biases in the automated decision-making process, it still relies on the trader's judgment for final approval, which might introduce human error or bias.

Key Differences Between the Related Works and the Proposed System

While these existing solutions provide useful insights into stock price prediction and trading automation, they each have limitations that our proposed system aims to address:

- **Control and Involvement:** Unlike the fully automated systems our system allows traders to approve trades based on AI-generated predictions, which helps maintain human control over the final decisions. This feature helps mitigate emotional biases that can influence human judgment and ensures that the trader remains involved in critical decision-making.
- **User-Friendliness:** Unlike some complex models, our system is designed to be accessible to both new and experienced traders. It includes a simple user interface that does not require advanced technical knowledge, making it usable for a broader audience.
- **Hybrid Approach:** While the models discussed in the related works focus primarily on stock price prediction or strategy optimization, our system offers a hybrid approach. It not only predicts stock prices but also provides buy/sell signals, enhancing decision-making and minimizing the risk of emotional bias in trading.

Chapter 3: Requirements and Analysis

3.1 Functional Requirements:

Functional requirements define what the system should do. These requirements focus on the features and functionality that the system provide to meet user needs.

- **Stock Prediction:** The system should be able to predict stock prices based on historical data using machine learning models like LSTM or Random Forest.
- **Buy/Sell Signals:** The system will generate buy/sell signals based on predictions, helping traders make decisions.
- **Trader Approval:** The system will allow traders to review and approve/reject the trade execution based on the generated signals.
- **Data Visualization:** Provide a user-friendly interface to visualize predictions, and trading signals.

3.2 Non-Functional Requirements:

Non-functional requirements describe the system's quality attributes, performance, and constraints.

- **Performance:** The system should provide predictions with minimal latency.
- **Usability:** The interface should be simple and user-friendly, requiring no advanced technical knowledge for use.
- **Reliability:** The system should provide accurate predictions consistently and function with minimal errors during its operation.
- **Scalability:** The system should be able to handle increased data volumes as the number of users or data inputs increases.
- **Security:** Ensure that the system has adequate security measures for user data and transactions, including user authentication.
- **Maintainability:** The system should be easy to update and maintain, ensuring long-term performance and feature updates.
- **Compatibility:** The system should work on different devices (e.g., web browsers, mobile phones) without issues.
- **Availability:** The system should be available 24/7, with minimal downtime for maintenance.

3.3 Software and Hardware Requirements:

This section will describe the tools and technologies required to implement the project, including both software and hardware.

Software Requirements:

1. **Programming Languages:** Python (for machine learning, data analysis, and web development)
2. **Machine Learning Libraries:** TensorFlow, Keras, scikit-learn, etc. (for building and training models)
3. **Web Framework:** Streamlit or Flask (for building the web-based user interface)
4. **Database:** MySQL, PostgreSQL, or SQLite (for storing historical stock data and user interaction data)
5. **Version Control:** Git (for version control and collaboration)
6. **Cloud Platforms:** Google Cloud Colab (for cloud-based storage, computation, and deployment)
7. **Visualization Tools:** Plotly or Matplotlib (for creating data visualizations)

Hardware Requirements:

1. **Local/Cloud Machine:** A computer/server with at least 8GB of RAM and a multi-core processor for machine learning model training.
2. **Graphics Processing Unit (GPU):** Recommended for training deep learning models like LSTM (if large-scale training is involved).
3. **Storage:** Sufficient storage for historical data (e.g., SSD or cloud-based storage) – at least 500GB depending on the volume of market data.
4. **Internet Connection:** High-speed internet connection for system access.

3.4 Initial System Design:

The initial system design is the blueprint for your system. It can include multiple types of diagrams, such as:

Use Case Diagram

This diagram shows the interactions between the **Trader** and the **System** in the AI-based stock trading system.

Actors:

- **Trader:** The user who interacts with the system to view predictions, generate signals, and approve/reject trades.
- **System:** Handles processes like stock price predictions, signal generation, and logging data.

Functions:

1. **Login/Register:** The trader logs in to the system.
2. **View Stock Price Predictions:** The trader views stock predictions generated by the system.
3. **Generate Buy/Sell Signals:** The system generates buy/sell signals based on predictions.
4. **Approve/Reject Trade Execution:** The trader approves or rejects the trade based on signals.
5. **Predict Stock Prices:** The system predicts stock prices using historical data.
6. **Monitor User Input:** The system tracks trader inputs for further actions.
7. **Generate Suggestions for Trade Execution:** The system provides suggestions for trade execution.
8. **Log Historical Data:** The system stores and logs market data.

Relationships:

- The **Trader** interacts with functions related to login, viewing predictions, and approving/rejecting trades.
- The **System** handles functions like generating signals, predicting stock prices, monitoring inputs, and logging data.
- **Generate Buy/Sell Signals** includes **Predict Stock Prices** (signals depend on predictions).
- **Approve/Reject Trade Execution** includes **Generate Buy/Sell Signals** (trade decisions depend on signals).
- **View Stock Price Predictions** includes **Generate Buy/Sell Signals** (signals are shown along with predictions).

This diagram illustrates how the trader and the system work together to manage stock predictions, trade signals, and trade decisions.

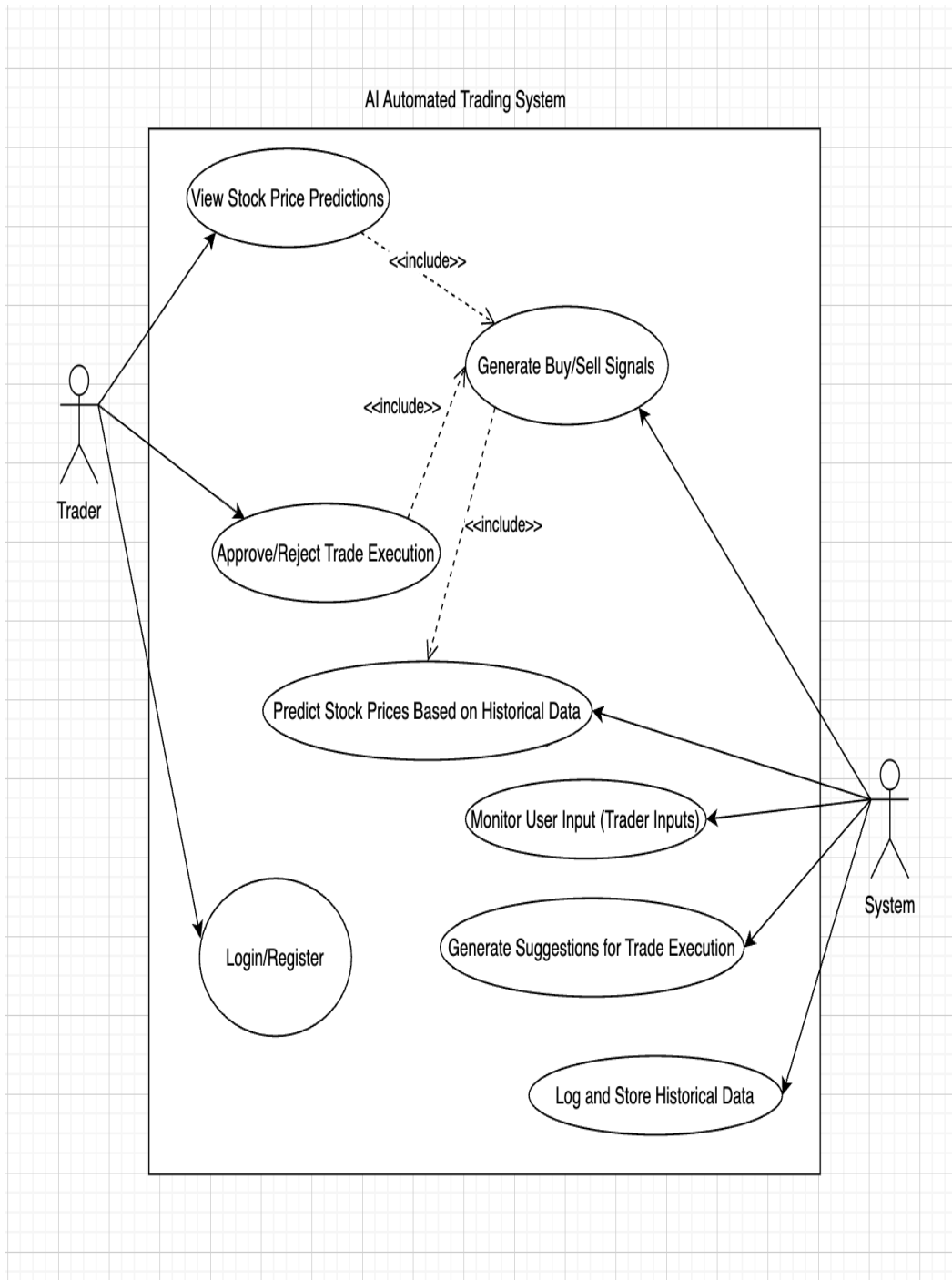


Figure 2: Use Case Diagram

Sequence Diagram

This sequence diagram illustrates the flow of requests and responses between the **Trader**, **UI**, **Controller**, **ML Model**, and **Database** as they work together to provide stock predictions, signals, and trade execution in the trading system.

Key Components and Their Roles:

1. **Trader**: The user (trader) interacting with the system.
2. **UI (User Interface)**: Displays the login status, predictions, signals, and trade confirmation to the trader.
3. **Controller**: Manages the system's logic, communicating with both the **Database** and the **ML Model**.
4. **ML Model**: The machine learning model used for predicting stock prices and generating buy/sell signals based on historical data.
5. **Database**: Stores and retrieves data, including user credentials, historical market data, and logs of generated signals and trades.

1. Login/Register

- The **Trader** interacts with the **UI** to log in or register.
- The **UI** sends the login request to the **Controller**.
- The **Controller** validates the credentials by querying the **Database**.
- The **Database** returns the validation result (success or failure).
- The **Controller** sends the login status back to the **UI**.
- The **UI** displays the login status to the **Trader**.

2. View Stock Price Predictions

- The **Trader** requests to view stock price predictions via the **UI**.
- The **UI** sends a request to the **Controller** for stock predictions.
- The **Controller** queries the **Database** for historical market data.
- The **Database** returns the historical data to the **Controller**.
- The **Controller** sends the data to the **ML Model** to make predictions about stock prices.

- The **ML Model** returns the predicted prices to the **Controller**.
- The **Controller** sends the predictions to the **UI**.
- The **UI** displays the stock predictions to the **Trader**.

3. Request Buy/Sell Signals

- The **Trader** requests buy/sell signals via the **UI**.
- The **UI** sends a request to the **Controller** to generate the buy/sell signals.
- The **Controller** sends the stock predictions to the **ML Model** for analysis.
- The **ML Model** returns the buy/sell signals to the **Controller**.
- The **Controller** logs the generated signals into the **Database**.
- The **Database** confirms the log.
- The **Controller** sends the signals back to the **UI**.
- The **UI** displays the buy/sell signals to the **Trader**.

4. Approve/Reject Trade

- The **Trader** approves or rejects the trade through the **UI**.
- The **UI** submits the trade decision to the **Controller**.
- The **Controller** logs the trade execution into the **Database**.
- The **Database** confirms the trade log.
- The **Controller** sends the trade status back to the **UI**.
- The **UI** displays the trade confirmation to the **Trader**.

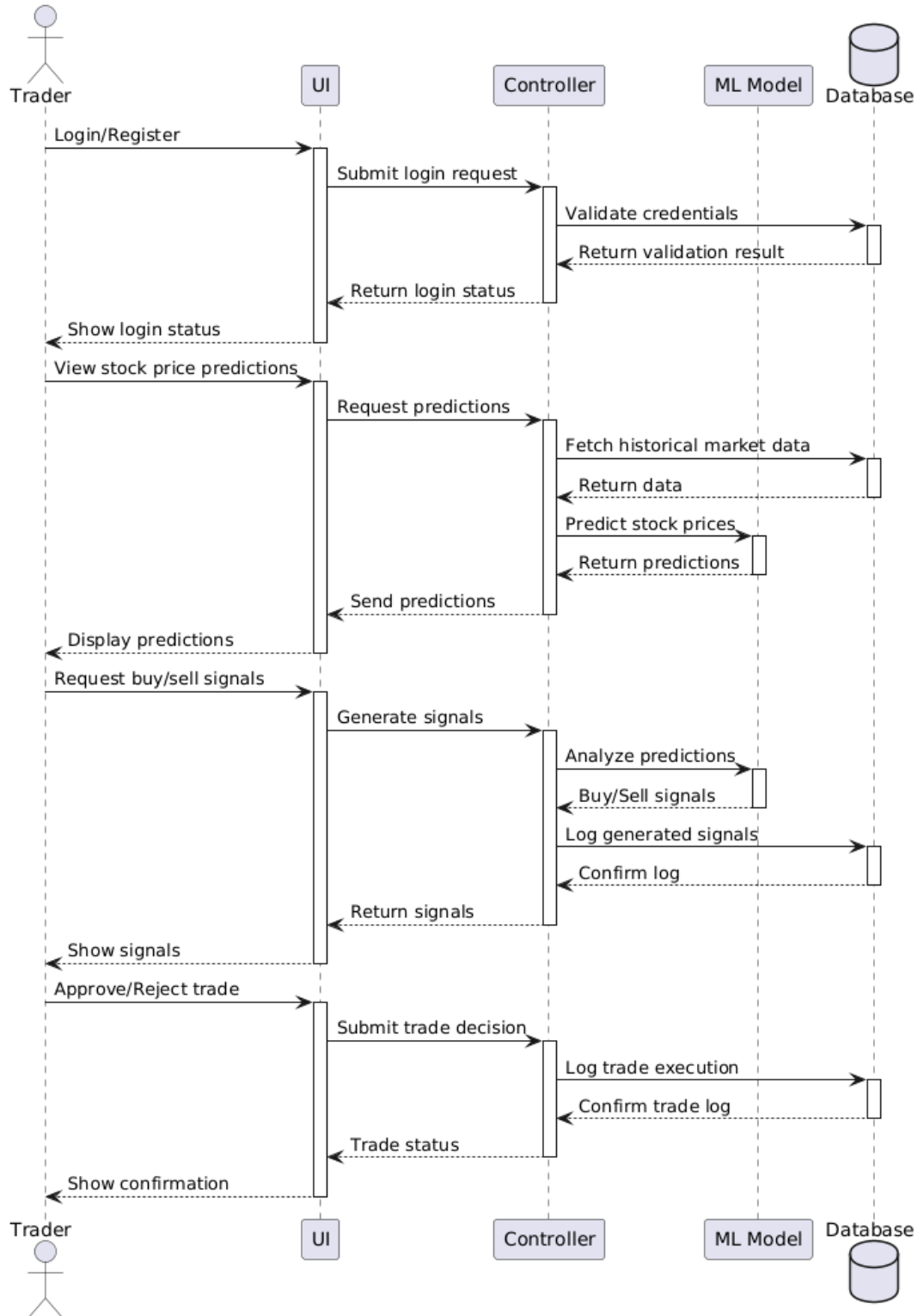


Figure 3: Sequence Diagram

Activity Diagram:

This **activity diagram** illustrates the workflow of an **AI Automated Stock Trading System** from login to trade execution.

1. **Login/Register:** The user starts by logging in or registering.
2. **Authenticate User:** The system authenticates the user's credentials.
 - If the authentication is successful, it proceeds to the next steps.
 - If the authentication fails, an error message is shown, and the process stops.
3. **Fetch Historical Data:** Upon successful login, the system fetches the required historical stock data.
4. **Display Stock Market:** The system displays the stock market data to the trader.
5. **Generate Predictions Using ML Model:** The system uses machine learning models to generate stock price predictions based on the historical data.
6. **Show Predictions & Buy/Sell Signals:** The generated predictions and buy/sell signals are displayed to the trader.
7. **Trader Approves/Rejects Trade:** The trader is then asked to approve or reject the suggested trade.
 - If the trader approves the trade, the system executes the trade, logs it in the database, and shows a trade confirmation.
 - If the trader rejects the trade, the system cancels the trade and shows a rejection message.
8. The process stops after the trade confirmation or rejection.

This diagram emphasizes the automated nature of the system, where the system generates signals and predictions, but the final decision on trade execution is left to the trader.

AI Automated Trading System - Activity Diagram

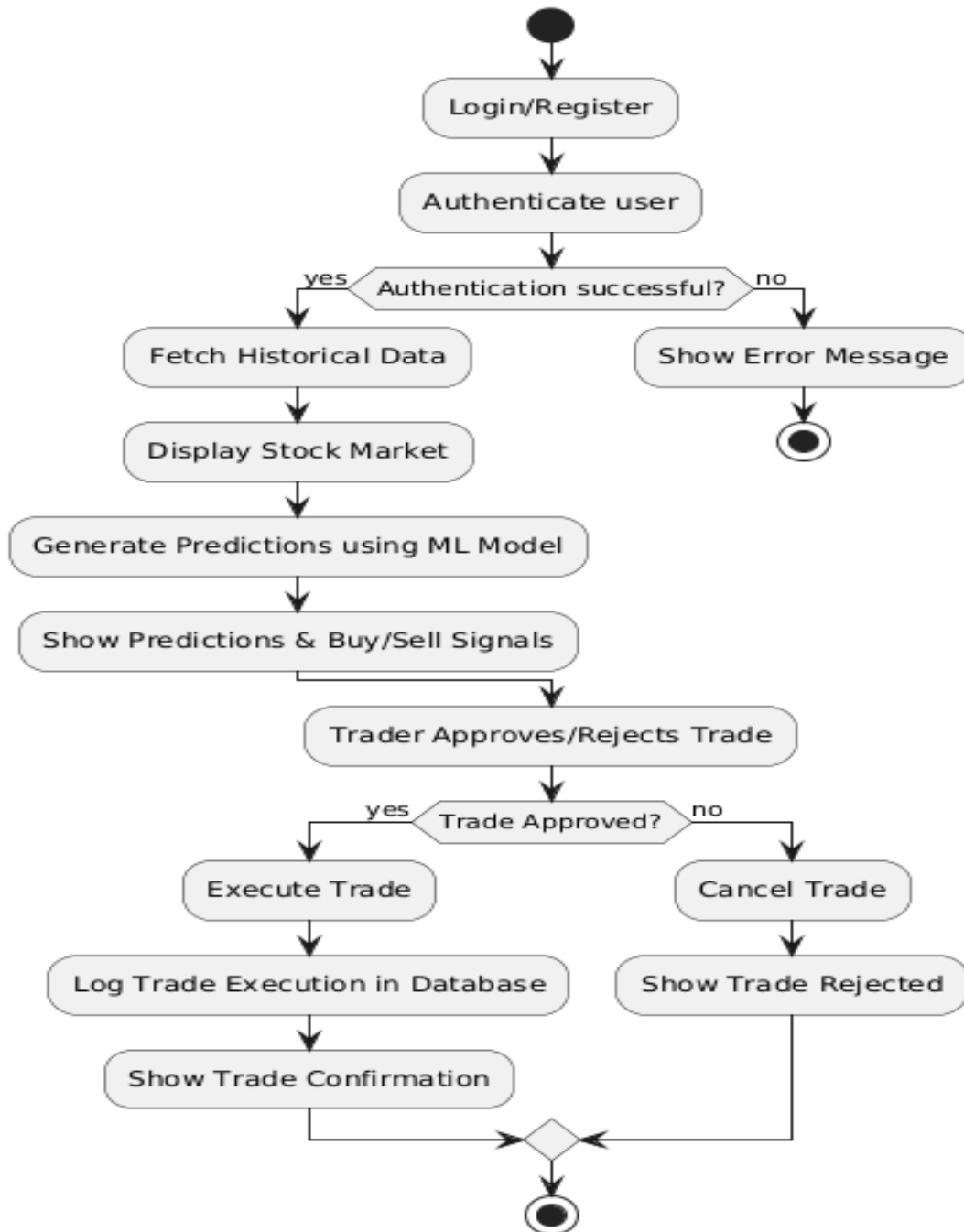


Figure 4: Activity Diagram

3.5 Feature insights and future enhancements

To be discussed in Part B

Chapter 4: Design, Implementation, and Testing

4.1 Overview

This chapter presents the design, implementation, and testing of the AI Automated Trading System.

It explains the software development approach used, describes the system architecture and implementation details, and discusses the changes that occurred during development.

The system was implemented as a **web-based application** using Python and Streamlit. It follows a **semi-automated trading approach**, where trading signals are generated by the system, while the final decision is made by the trader through an approval or rejection process (Human-in-the-Loop).

4.2 Software Development Approach

4.2.1 Used Approach

The **Agile development approach** was used in this project.

The system was developed incrementally through small, independent modules such as:

- User authentication
- Dashboard and data visualization
- Stock price prediction
- Signal generation
- Signal approval and management

Each module was implemented and tested separately before being integrated into the full system.

4.2.2 Justification of the Used Approach

Agile was chosen because:

- System requirements evolved during development
- Machine learning models required continuous testing and refinement
- User interface design needed frequent improvements

Agile allowed flexibility, faster feedback, and easier handling of changes compared to rigid development models.

4.2.3 Phases of the Chosen Model Approach

The Agile development process followed these phases:

1. **Requirements Analysis**
Identification of functional and non-functional requirements such as prediction accuracy, usability, and system performance.
2. **System Design**
Design of system pages, services, controllers, and database structure.
3. **Implementation**
Development of the system using Python, Streamlit, and machine learning models.
4. **Testing**
Testing individual modules and integrated system components.
5. **Refinement and Improvement**
Updating the user interface, adjusting prediction parameters, and improving signal logic.

4.2.4 Why It Is Better Than Other Approaches

Compared to the Waterfall model, Agile is more suitable for AI-based systems because:

- Requirements are not fixed from the beginning
- Models need iterative improvement
- UI changes can be applied easily

This makes Agile more flexible and practical for machine learning applications.

4.2.5 Alternative Approach

An alternative approach could be the **Waterfall model**, where development progresses sequentially through fixed phases.

However, this approach is less effective for AI systems due to limited flexibility and difficulty in handling changes during development.

4.3 Changes in Design

Several design changes were introduced during the implementation phase to improve system usability, accuracy, and safety.

4.3.1 Changes in the Functional Requirements

No major changes

4.3.2 Changes in the Non-Functional Requirements

Aspect	Change
User Interface	Full dark mode added
Usability	Interactive sliders and tables
Performance	Modular MVC architecture

These changes improved user experience and system responsiveness.

4.3.3 Changes in Diagrams

Although the core functionalities of the AI Automated Trading System remained unchanged throughout the project, minor differences appear between the naming conventions used in the design diagrams and those used in the implementation code.

During the early design phase, function and interaction names were defined at a **conceptual level** to clearly describe system behavior. These names were intentionally generic to improve readability and understanding in diagrams such as Use Case, Sequence, and Activity Diagrams.

Design-Level Name	Implementation-Level Name	Functionality
Generate Signal	<code>predict() / log_signal()</code>	Automatic signal generation
Approve Signal	<code>approve()</code>	User approval
Reject Signal	<code>reject()</code>	User rejection
View Signals	<code>list_pending()</code>	Display signals

4.3.4 Changes in Software Requirements

There is no changes in Software Requirements.

4.3.5 Software Modeling Tools Used

The following tools were used in system design and development:

- **Streamlit** for user interface development
- **Plotly** for interactive charts
- **SQLite** for signal storage

4.3.6 Specifications That Distinguish This Software from Other Software

The AI Automated Trading System includes several distinguishing features:

1. **Hybrid Prediction Approach**
Combines multiple machine learning techniques to improve prediction quality.
2. **Human-in-the-Loop Design**
Trading signals require trader approval before execution.
3. **Interactive and User-Friendly Interface**
Includes sliders, tables, and real-time charts.
4. **Modular System Architecture**
Separation between pages, controllers, services, and models.

These features make the system more reliable, scalable, and suitable for real-world trading support.

4.4 Implementation

4.4.1 User Interface

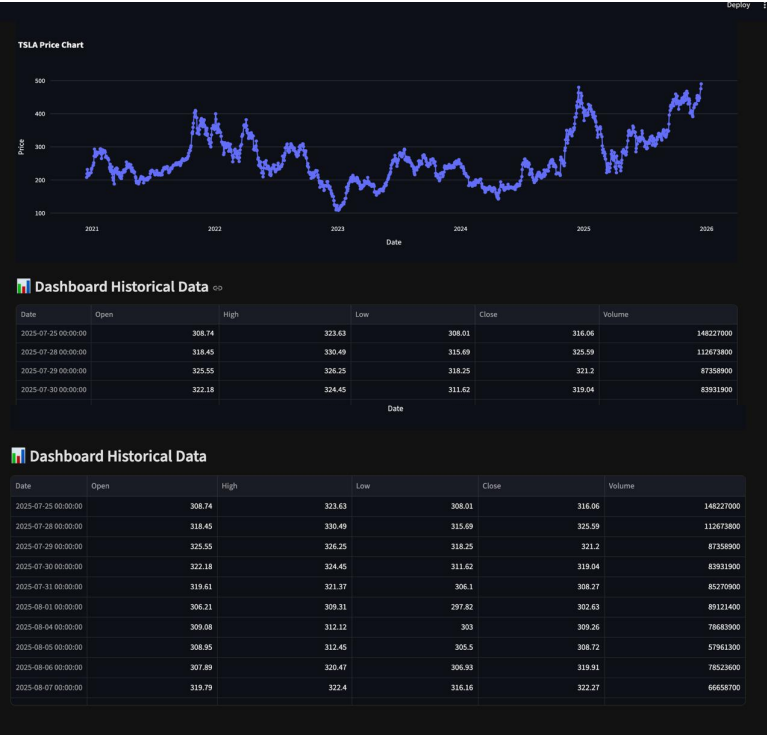
The user interface is implemented using **Streamlit**, providing an interactive web-based application that follows a dark-mode theme with responsive layout and intuitive navigation via tabs. Key features include:

- **Splash Screen:** Displays the project logo at startup welcoming users before accessing the app.
 -

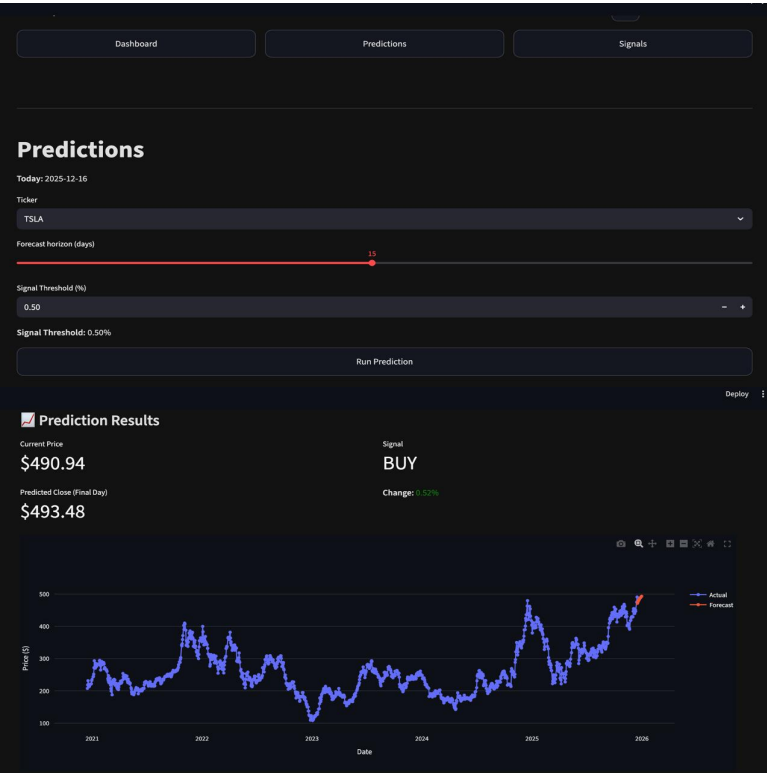


- **Authentication:** Users can **login or register** via a secure interface. Credentials are stored in the **SQLite** database.

- **Navigation Tabs:** Users can switch between **Dashboard**, **Predictions**, and **Signals** pages.
- **Dashboard Page:** Displays historical stock data in a **Plotly** chart and **tabular** format for the last 100 entries.



- **Predictions Page:** Users can select a **stock ticker** and forecast horizon. The **hybrid ML model** runs predictions, displaying **future prices**, signals, and metrics.





- Signals Page:** Pending signals can be **approved or rejected** with dynamic status badges. Historical signals are also displayed. Signals have a lifecycle: PENDING → APPROVED/REJECTED

Ticker	Type	Status	Approve	Reject
TSLA	BUY	PENDING	Approve	Reject
TSLA	BUY	APPROVED	—	—
GOOG	SELL	REJECTED	—	—
AMD	SELL	APPROVED	—	—

4.4.2 Code

The project is structured using a **modular MVC [Model-View-Controller] architecture**:

- Controllers:** Handle business logic and communication between UI and backend.
 - auth_controller.py*: Login and registration functions.
 - data_controller.py*: Fetch historical stock data.
 - prediction_controller.py*: Run LSTM + Random Forest hybrid predictions.
 - trade_controller.py*: Manage signal lifecycle (approve/reject/list).
- Pages:** Streamlit pages handle rendering (*login_page.py*, *dashboard_page.py*, *predictions_page.py*, *signals_page.py*).
- Components:** Reusable UI elements and charts (*charts.py*, *layout.py*).
- ML Pipeline:**
 - LSTM Model:** Predicts future stock prices from historical sequences.
 - Random Forest Residuals:** Refines predictions to improve test performance.

- **Evaluation:** Metrics such as **MAE**, **RMSE**, and **R²** are calculated.
- **Helpers:** Base64 Logo encoding, dark mode styling, and session state management.

4.4.3 Database

The system uses **SQLite** for persistent storage:

- **Database Connection:** *database/db.py* initializes and manages the connection.
- **Migrations:** *migrations.sql* creates necessary tables if they do not exist.
- **Database Schema:**
 - **users:** Stores usernames and hashed passwords for authentication.
 - **signals:** Stores trading signals with columns for **ticker**, **type**, **confidence**, **status**, and **created_at**.
- **Database Access:** Controllers interact with the database through dedicated functions, ensuring separation of concerns.
- **Data Handling**
 - Stock data is **fetches from yFinance** using *data_controller.py*
 - Historical prices are processed and stored temporarily for analysis.
 - Signals are generated by the prediction pipeline and stored in the signals table.

4.5 Issues and Problems Faced in the Project

During the development of the AI Automated Trading System, several challenges were encountered:

- **Data Availability & Quality**
 - Missing or inconsistent stock market data for certain tickers.
 - Variations in yFinance API responses occasionally caused failed fetch attempts.
- **Model Performance**
 - Market volatility caused fluctuations in prediction metrics
 - Initial LSTM predictions had high error on test data.
 - Hybrid approach with Random Forest residuals was implemented to enhance performance.
- **Real-Time UI Updates**
 - Streamlit's session management required careful handling to maintain user state across tabs.
 - Splash screen and dynamic tab navigation needed precise state control.
- **Database Management**
 - Signal lifecycle implementation (PENDING → APPROVED/REJECTED) needed robust handling.
 - Concurrent access to SQLite (single-threaded) required disabling `check_same_thread`.

- **Integration Challenges:**
 - Synchronizing ML predictions with Streamlit UI and database required careful management of **session state** and button callbacks.

4.6 Testing

A combination of white box, black box, unit, system, and acceptance testing was applied to ensure correctness and usability.

4.6.1 White Box Testing

- Internal functions were tested for logic correctness, including *run_prediction()*, *data preprocessing*, and *database CRUD operations*.

4.6.2 Black Box Testing

- The application was tested from a user perspective for usability, functionality, and UI responsiveness.
- Tested user flows: login, registration, dashboard, predictions, and signal approval/rejection.
- Ensured UI elements rendered correctly without inspecting internal code.
- Confirmed predictions and signals behaved as expected for different tickers.

4.6.3 Unit Testing

- Individual components like *chart rendering*, *login validation*, and *signal approval/rejection* were tested independently.
 - *auth_controller.py*: Valid/invalid login and registration.
 - *trade_controller.py*: Approve/reject signals correctly updated the database.
 - *prediction_controller.py*: Prediction outputs matched expected dimensions and ranges.

4.6.4 System Testing

- End-to-end testing verified proper interaction between ML models, database, and UI pages.

4.6.5 Acceptance Testing

- User tests confirmed that the system met functional requirements for stock prediction and signal management.

4.6.6 Validation and Verification

- Predictions were validated on **test data**, comparing *MAE*, *RMSE*, and *R²* with historical trends.
- Verification ensured that **functional and non-functional requirements** were met.

4.7 Summary of the Chapter

- Successfully implemented a modular, MVC-based AI trading system.
- Developed a hybrid ML prediction pipeline for stock forecasting.
- Implemented user authentication, dynamic dashboards, prediction pages, and a signal lifecycle system.
- Applied comprehensive testing to ensure functionality, correctness, and usability.
- Key lessons learned included **database handling, session management, and hybrid ML model integration**.

Chapter 5: Results and Discussion

5.1 Overview of This Chapter

This chapter explains the final results of the project and discusses what was achieved during the development of the AI-based trading system. It focuses on how the system performed, the skills gained during the project, and whether the project goals were successfully met. In addition, this chapter discusses future improvements and features that could not be completed due to time limitations. The purpose of this chapter is to evaluate the overall success of the project and reflect on the learning outcomes.

5.2 Critical Discussion

The developed system successfully demonstrates how artificial intelligence techniques can be used to support stock trading decisions. The system was able to collect historical stock data, process it, apply machine learning models, and generate buy and sell signals. The results show that the system can identify trends and provide useful predictions based on past market behavior.

However, the system is not perfect. Stock markets are highly volatile and affected by many external factors such as news, economic events, and investor emotions. These factors are difficult to fully capture using historical data alone. As a result, some predictions may not always match real market movements. To

reduce risk, the system was designed as a **semi-automated system**, where the final trading decision is approved by the trader.

Overall, the results are acceptable for an academic project and prove that AI can assist traders, but it should not be fully relied on for real-world trading without further improvements and testing.

5.3 Skills Acquired from the Project

During the development of this project, several important technical and professional skills were acquired, including:

- Understanding how AI-based systems are designed and developed
- Working with historical financial data and preprocessing it
- Applying machine learning models for prediction tasks
- Developing a web-based user interface for system interaction
- Integrating backend logic with frontend visualization
- Improving problem-solving and debugging skills
- Managing time and project tasks effectively

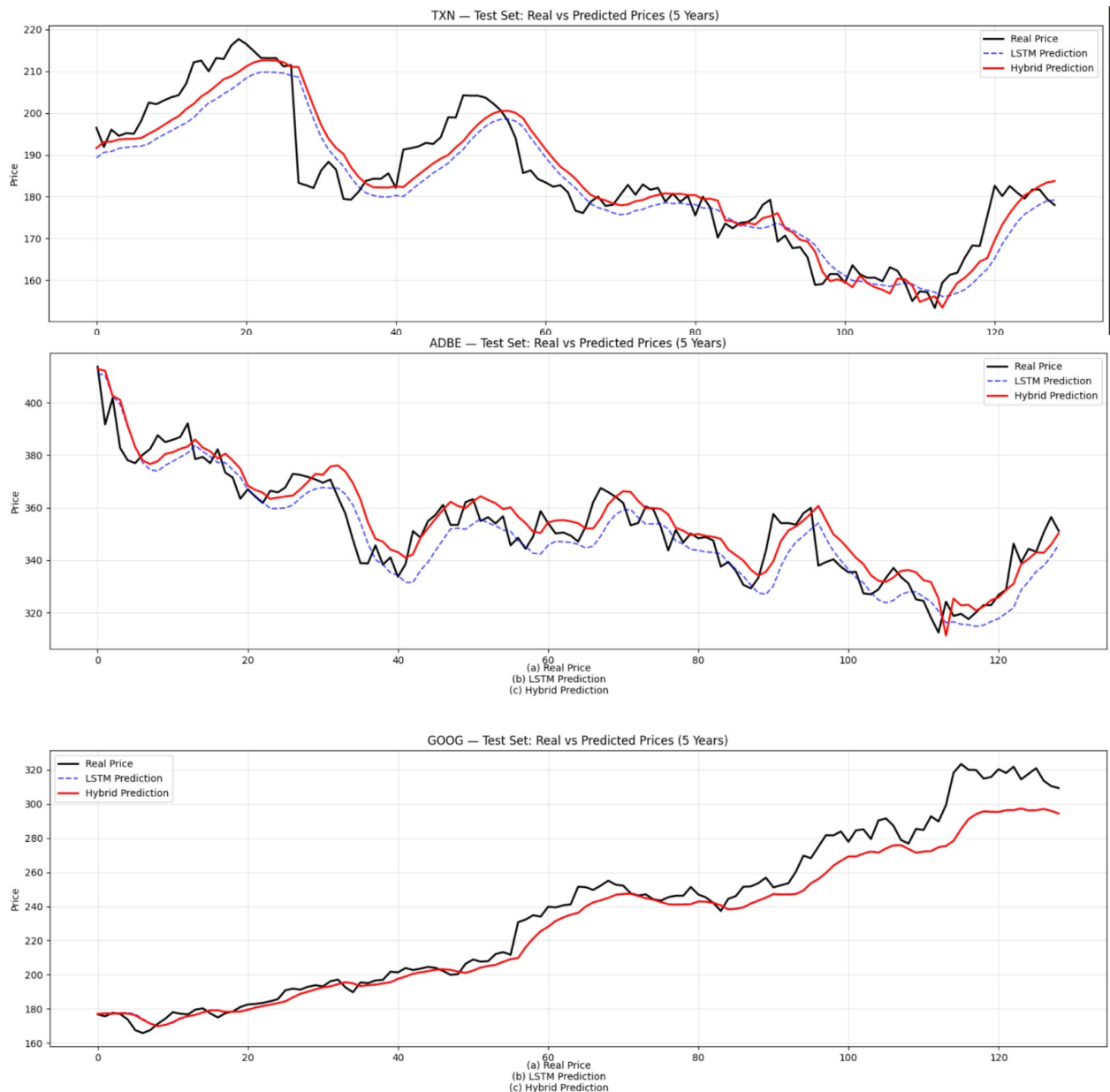
These skills are valuable for future projects and professional work in software development and artificial intelligence.

5.4 Development Goals and Results

5.4.1 Findings and Results

The main findings of this project show that combining data analysis with machine learning techniques can produce meaningful trading signals. The system was able to predict price movements within a reasonable range and visualize results clearly for the user. Performance evaluation metrics showed that the predictions were reasonably accurate for short-term forecasting, especially when using cleaned and well-prepared data.

The system also successfully allowed traders to review predictions before taking action, which increases trust and reduces risk.





5.4.2 Project Goals

The original goals of the project were mostly achieved. These goals included:

- Designing an AI-based trading assistance system
- Implementing a prediction mechanism using historical data
- Creating a user-friendly interface
- Supporting trader decision-making instead of full automation

All these goals were met within the project timeline, making the project successful from both technical and academic perspectives.

	Ticker	LSTM_Val_R2	LSTM_Test_R2	LSTM_Test_MAE	LSTM_Test_RMSE	LSTM_Test_DirAcc	Hybrid_Val_R2	Hybrid_Test_R2	Hybrid_Test_MAE	Hybrid_Test_RMSE
0	ADBE	0.852174	0.792966	6.907156	8.739049	0.476562	0.857017	0.817781	6.308428	8.198591
1	AMD	0.952076	0.944258	6.451452	9.307301	0.500000	0.953070	0.942379	6.711031	9.462913
2	GOOG	0.899672	0.936923	8.469208	11.698816	0.546875	0.900790	0.935844	8.569512	11.798480
3	SNPS	0.502494	0.563679	45.723956	52.013478	0.460938	0.471385	0.692450	34.194203	43.668774
4	TSLA	0.925161	0.918886	12.900509	16.115115	0.507812	0.920697	0.914865	13.495955	16.509769
5	TXN	0.560220	0.817727	5.527522	6.974247	0.484375	0.644994	0.850529	4.646718	6.315595

5.5 Future Work

5.5.1 New Areas of Investigation

Several enhancements can be implemented in future versions of the system:

1. **Integration of Real-Time Data**
Using live market data to improve prediction accuracy.
2. **Additional Technical Indicators**
Including more indicators to enhance feature representation.
3. **Risk Management Features**
Adding stop-loss and take-profit mechanisms.
4. **Model Optimization**
Improving model performance through further tuning and evaluation.
5. **User Notifications**
Sending alerts when new trading signals are generated.
6. **Expanded Asset Support**
Supporting more stocks and financial instruments.

These improvements would enhance the system's reliability, scalability, and real-world applicability.

5.5.2 Features Not Completed Due to Time Constraints

Some features were planned but could not be completed due to limited time, including:

- Full real-time trading execution
- Advanced risk management tools
- Mobile application support

- Automated alerts via email or mobile notifications

These features can be implemented in future versions of the system.

5.6 Summary of the Chapter

This chapter presented the results and discussion of the AI-based trading system project. It evaluated system performance, discussed strengths and limitations, and reflected on the skills gained during development. The chapter also reviewed project goals and suggested future improvements. Overall, the project achieved its main objectives and provides a strong foundation for further development and research.

Key improvements:

- Maintains human control while automating analysis.
- More accurate predictions using hybrid ML models.
- User-friendly interface increases accessibility for novice and experienced traders.

Chapter 6: Conclusion

6.1 Conclusion

This project presented the design and implementation of an **AI Automated Trading System** that supports traders in making informed investment decisions. The system analyzes historical stock market data and generates Buy/Sell signals using machine learning techniques, while keeping the trader involved in the final decision through a **Human-in-the-Loop** approach.

The system was implemented as a **web-based application** using Python and Streamlit, providing an interactive and user-friendly interface.

Key features include stock price visualization, multi-day price forecasting, adjustable signal thresholds, and a signal approval mechanism.

The project successfully achieved its main objectives:

- Automating stock data analysis
- Generating trading signals based on predictions
- Reducing emotional and rushed trading decisions
- Maintaining human control over final trade actions

Overall, the developed system demonstrates how artificial intelligence can be effectively used to support trading decisions while minimizing risk through user approval.

6.2 System Limitations

Despite the successful implementation, the system has some limitations:

- The system relies on historical market data, which may not fully reflect sudden market changes.
- External factors such as news events and economic indicators are not included.
- The system supports a limited number of predefined stocks.
- Trade execution is not fully automated and depends on user approval.

These limitations are acceptable for an academic project but can be addressed in future improvements.

6.3 Final Remarks

The AI Automated Trading System demonstrates the practical application of artificial intelligence in financial decision support systems.

By combining machine learning with human oversight, the system provides a balanced and responsible approach to automated trading.

This project forms a solid foundation for future research and development in intelligent trading systems.

References

1. Adarsh, A. (n.d.) *Stock Price Prediction Using LSTM*. GitHub repository. Available at: <https://github.com/034adarsh/Stock-Price-Prediction-Using-LSTM> (Accessed: 5 May 2025).
2. Amritpal-001 (n.d.) *Stock Price Prediction Using LSTM*. GitHub repository. Available at: <https://github.com/Amritpal-001/Stock-price-prediction> (Accessed: 5 May 2025).
3. draw.io (n.d.) *diagrams.net*. Available at: <https://app.diagrams.net/> (Accessed: 5 May 2025).
4. Investopedia (n.d.) *AI in Trading*. Available at: <https://www.investopedia.com/> (Accessed: 5 May 2025).
5. Jatin-Goyal-552 (n.d.) *Stock Price Prediction*. GitHub repository. Available at: https://github.com/Jatin-Goyal-552/Stock_Price_Predicton (Accessed: 5 May 2025).
6. Li, Z., Yu, H., Xu, J., Liu, J. and Mo, Y. (2023) 'Stock market analysis and prediction using LSTM: A case study on technology stocks', *Innovations in Applied Engineering and Technology*, 2(1), pp. 1–6. Available at: <https://doi.org/10.62836/iaet.v2i1.162>.
7. Liu, J., et al. (2021) 'AI-based hybrid stock trading system using Random Forest and Support Vector Machines', *Financial Innovation*. Available at: <https://link.springer.com/article/10.1186/s40854-021-00288-1> (Accessed: 5 May 2025).
8. Medium (n.d.) *Articles about AI trading*. Available at: <https://medium.com/> (Accessed: 5 May 2025).
9. Mendoza, L. and Zhang, M. (2019) 'Design of an automated stock trading system using artificial intelligence', *International Journal of Financial Engineering*. Available at: <https://www.tandfonline.com> (Accessed: 5 May 2025).
10. Moghar, A. and Hamiche, M. (2020) 'Stock market prediction using LSTM recurrent neural network', *Procedia Computer Science*, 176, pp. 2766–2775. Available at: <https://doi.org/10.1016/j.procs.2020.09.319>.
11. PlantUML (n.d.) *PlantUML*. Available at: <https://plantuml.com/> (Accessed: 5 May 2025).
12. Python Software Foundation (n.d.) *Python*. Available at: <https://www.python.org/> (Accessed: 5 May 2025).
13. Scikit-learn (n.d.) *Scikit-learn: Machine learning in Python*. Available at: <https://scikit-learn.org/> (Accessed: 5 May 2025).
14. Sinan W. (n.d.) *LSTM Stock Price Prediction*. GitHub repository. Available at: <https://github.com/sinanw/lstm-stock-price-prediction> (Accessed: 5 May 2025).
15. Yahoo Finance (n.d.) *Financial market data*. Available at: <https://www.yahoo.com/finance> (Accessed: 5 May 2025).
16. Kaggle (n.d.) *Datasets for machine learning*. Available at: <https://www.kaggle.com/>